

EVIDENCIA AVANCE 1

Código Fuente

Main

```
1 //===== main.cpp =====//
2
3 #include <iostream>
4 #include <vector>
5
6 #include "productos.h"
7 #include "ventas.h"
8 #include "reportes.h"
9 #include "utilidades.h"
10
11 using namespace std;
12
13 //===== VECTOR MENU =====//
14
15 vector<string> opcionesMenu = {
16     "1. Gestion de Productos",
17     "2. Proceso de Venta",
18     "3. Reportes",
19     "4. Utilidades",
20     "5. Salir"
21 };
22
23
24 //===== MENU PRINCIPAL =====//
25
26 void MenuPrincipal() {
27     Encabezado("SISTEMA DE INVENTARIO Y VENTAS");
28
29     for(size_t i = 0; i < opcionesMenu.size(); i++) {
30         cout << opcionesMenu[i] << endl;
31     }
32
33     cout << "\nSeleccione una opcion: ";
34 }
35
36
37
```

```
38 //===== MAIN =====//
39
40 int main() {
41     int opcion;
42
43     do {
44         MenuPrincipal();
45
46         cin >> opcion;
47
48         if(cin.fail()) {
49             cin.clear();
50             cin.ignore(1000, '\n');
51             opcion = 0;
52         }
53
54         switch(opcion) {
55             case 1:
56                 MenuProductos();
57                 break;
58             case 2:
59                 CrearVenta();
60                 break;
61             case 3:
62                 MenuReportes();
63                 break;
64         }
65     } while(opcion != 5);
66
67     return 0;
68 }
```

```

75         case 4:
76             MenuUtilidades();
77             break;
78
79         case 5:
80
81             cout << "\nSistema finalizado.\n";
82             break;
83
84         default:
85
86             cout << "\nOpcion invalida.\n";
87
88     }
89
90     if(opcion != 5) {
91         PausarSistema();
92         LimpiarPantalla();
93     }
94 } while(opcion != 5);
95
96 return 0;
97
98 }

```

Productos archivo h

```

C Productos.h > ...
1 //===== productos.h =====//
2
3 #ifndef PRODUCTOS_H
4 #define PRODUCTOS_H
5
6 #include <vector>
7
8 using namespace std;
9
10 //===== ESTRUCTURA PRODUCTO =====//
11
12 struct Producto {
13
14     int codigo;
15     char nombre[40];
16     float precio;
17     int stock;
18     int vendidos;
19     bool activo;
20 };
21
22 //===== MENU =====//
23
24 void MenuProductos();
25
26 //===== CRUD =====//
27
28 void RegistrarProducto();
29 void MostrarProductos();
30
31 void BuscarPorCodigo();
32 void BuscarPorNombre();
33
34 void ActualizarStock();
35 void ModificarPrecio();
36
37 void EliminarProducto();
38
39 //===== ORDENAMIENTOS =====//
40
41 void OrdenarPorPrecio(); // Bubble Sort
42 void OrdenarPorStock(); // Selection Sort
43 void OrdenarPorVendidos(); // Bubble Sort
44
45 //===== UTILIDADES =====//
46
47 bool ExisteProducto(int codigo);
48
49 vector<Producto> CargarProductos();
50
51 #endif

```

Productos archivo .cpp

```
Products.cpp > ...
1 //===== productos.cpp =====//
2
3 #include <iostream>
4 #include <fstream>
5 #include <iomanip>
6 #include <vector>
7 #include <cstring>
8
9 #include "productos.h"
10 #include "utilidades.h"
11
12 using namespace std;
13
14 const char* archivoProductos = "productos.dat";
15
16 //===== CARGAR PRODUCTOS =====//
17
18 vector<Producto> CargarProductos() {
19
20     vector<Producto> productos;
21
22     Producto p;
23
24     ifstream archivo(
25         archivoProductos,
26         ios::binary
27     );
28
29     if(!archivo) {
30
31         return productos;
32     }
33
34     while(
35         archivo.read(
36             (char*)&p,
37             sizeof(Producto))
38     ) {
39
40         if(p.activo) {
41
42             productos.push_back(p);
43         }
44     }
45
46     archivo.close();
47
48     return productos;
49 }
50
51 //===== EXISTE PRODUCTO =====//
52
53 bool ExisteProducto(int codigo) {
54
55     Producto p;
56
57     ifstream archivo(
58         archivoProductos,
59         ios::binary
60     );
61
62     if(!archivo) {
63
64         return false;
65     }
66
67     while(
68         archivo.read(
69             (char*)&p,
70             sizeof(Producto))
71     ) {
```

```

72         if(
73             p.codigo == codigo &&
74             p.activo
75         ) {
76             archivo.close();
77             return true;
78         }
79     }
80     return false;
81 }
82
83 archivo.close();
84
85 return false;
86 }
87
88 //===== MENU PRODUCTOS =====//
89
90 void MenuProductos() {
91     int opcion;
92
93     do {
94         Encabezado(
95             "GESTION DE PRODUCTOS"
96         );
97
98         cout << "1. Registrar producto\n";
99         cout << "2. Mostrar productos\n";
100         cout << "3. Buscar por codigo\n";
101         cout << "4. Buscar por nombre\n";
102         cout << "5. Actualizar stock\n";
103
104         cout << "6. Modificar precio\n";
105         cout << "7. Eliminar producto\n";
106         cout << "8. Ordenar por precio\n";
107         cout << "9. Ordenar por stock\n";
108         cout << "10. Ordenar por vendidos\n";
109         cout << "11. Regresar\n";
110
111         cout << "\nSeleccione opcion: ";
112         cin >> opcion;
113
114         if(cin.fail()) {
115             cin.clear();
116             cin.ignore(1000, '\n');
117
118             opcion = 0;
119         }
120
121         switch(opcion) {
122             case 1:
123                 RegistrarProducto();
124                 break;
125
126             case 2:
127                 MostrarProductos();
128                 break;
129
130             case 3:
131                 BuscarPorCodigo();
132                 break;
133
134             case 4:
135                 BuscarPorNombre();
136
137
138
139

```

```

140         break;
141
142         case 5:
143             ActualizarStock();
144             break;
145
146         case 6:
147             ModificarPrecio();
148             break;
149
150         case 7:
151             EliminarProducto();
152             break;
153
154         case 8:
155             OrdenarPorPrecio();
156             break;
157
158         case 9:
159             OrdenarPorStock();
160             break;
161
162         case 10:
163             OrdenarPorVendidos();
164             break;
165     }
166
167     } while(opcion != 11);
168 }
169
170 //===== REGISTRAR PRODUCTO =====//
171

```

```

172 void RegistrarProducto() {
173     Producto p;
174
175     cout << "\nCodigo: ";
176     cin >> p.codigo;
177
178     if(cin.fail()) {
179
180         cin.clear();
181         cin.ignore(1000, '\n');
182
183         cout << "Codigo invalido.\n";
184         return;
185     }
186
187     if(ExisteProducto(p.codigo)) {
188
189         cout << "El codigo ya existe.\n";
190         return;
191     }
192
193     cin.ignore();
194
195     cout << "Nombre: ";
196     cin.getline(p.nombre, 40);
197
198     cout << "Precio: ";
199     cin >> p.precio;
200
201     if(cin.fail() || p.precio <= 0) {
202
203         cin.clear();
204         cin.ignore(1000, '\n');
205
206         cout << "Precio invalido.\n";
207         return;
208     }

```

```

209     }
210
211     cout << "Stock: ";
212     cin >> p.stock;
213
214     if(cin.fail() || p.stock < 0) {
215         cin.clear();
216         cin.ignore(1000, '\n');
217
218         cout << "Stock invalido.\n";
219         return;
220     }
221
222     p.vendidos = 0;
223     p.activo = true;
224
225     ofstream archivo(
226         archivoProductos,
227         ios::binary | ios::app
228     );
229
230     if(!archivo) {
231         cout << "Error al abrir archivo.\n";
232         return;
233     }
234
235     archivo.write(
236         (char*)&p,
237         sizeof(Producto)
238     );
239
240     archivo.close();
241
242     cout << "\nProducto registrado correctamente.\n";
243 }
244
245 //===== MOSTRAR PRODUCTOS =====//
246
247 void MostrarProductos() {
248
249     vector<Producto> productos =
250     CargarProductos();
251
252     if(productos.empty()) {
253         cout << "\nNo existen productos registrados.\n";
254         return;
255     }
256
257     Encabezado("LISTA DE PRODUCTOS");
258
259     FormatoDecimal();
260
261     cout << left
262         << setw(10) << "Codigo"
263         << setw(25) << "Nombre"
264         << setw(12) << "Precio"
265         << setw(10) << "Stock"
266         << setw(10) << "Vendidos"
267         << endl;
268
269     cout << "-----\n";
270
271     for(size_t i = 0;
272         i < productos.size();
273         i++) {

```

```

277         cout << left
278             << setw(10)
279             << productos[i].codigo
280
281             << setw(25)
282             << productos[i].nombre
283
284             << setw(12)
285             << productos[i].precio
286
287             << setw(10)
288             << productos[i].stock
289
290             << setw(10)
291             << productos[i].vendidos
292
293             << endl;
294     }
295 }
296
297 //===== BUSCAR POR CODIGO =====//
298
299 void BuscarPorCodigo() {
300
301     vector<Producto> productos =
302     CargarProductos();
303
304     if(productos.empty()) {
305
306         cout << "\nNo existen productos registrados.\n";
307         return;
308     }
309 }
310

```

```

311 int codigo;
312 bool encontrado = false;
313
314 cout << "\nIngresa codigo: ";
315 cin >> codigo;
316
317 if(cin.fail()) {
318
319     cin.clear();
320     cin.ignore(1000, '\n');
321
322     cout << "Codigo invalido.\n";
323     return;
324 }
325
326 for(size_t i = 0;
327     i < productos.size();
328     i++) {
329
330     if(productos[i].codigo == codigo) {
331
332         encontrado = true;
333
334         Encabezado("PRODUCTO ENCONTRADO");
335
336         FormatoDecimal();
337
338         cout << "Codigo: "
339             << productos[i].codigo
340             << endl;
341
342         cout << "Nombre: "
343             << productos[i].nombre
344             << endl;
345

```

```

346         cout << "Precio: Q"
347             << productos[i].precio
348             << endl;
349
350         cout << "Stock: "
351             << productos[i].stock
352             << endl;
353
354         cout << "Vendidos: "
355             << productos[i].vendidos
356             << endl;
357
358         break;
359     }
360 }
361
362 if(!encontrado) {
363
364     cout << "\nProducto no encontrado.\n";
365 }
366 }
367
368 //===== BUSCAR POR NOMBRE =====//
369
370 void BuscarPorNombre() {
371
372     vector<Producto> productos =
373         CargarProductos();
374
375     if(productos.empty()) {
376
377         cout << "\nNo existen productos registrados.\n";
378         return;

```

```

379     }
380
381     char nombreBuscar[40];
382
383     bool encontrado = false;
384
385     cin.ignore();
386
387     cout << "\nIngrese nombre: ";
388     cin.getline(nombreBuscar,40);
389
390     Encabezado("RESULTADOS BUSQUEDA");
391
392     for(size_t i = 0;
393         i < productos.size();
394         i++) {
395
396         if(
397             strstr(
398                 productos[i].nombre,
399                 nombreBuscar
400             )
401         ) {
402
403             encontrado = true;
404
405             cout << "\nCodigo: "
406                 << productos[i].codigo
407                 << endl;
408
409             cout << "Nombre: "
410                 << productos[i].nombre
411                 << endl;
412
413             cout << "Precio: Q"

```



```

414         << productos[i].precio
415         << endl;
416
417         cout << "Stock: "
418         << productos[i].stock
419         << endl;
420     }
421 }
422
423 if(!encontrado) {
424
425     cout << "\nNo se encontraron coincidencias.\n";
426 }
427 }
428
429 //===== ACTUALIZAR STOCK =====//
430
431 void ActualizarStock() {
432
433     Producto p;
434
435     int codigo;
436     bool encontrado = false;
437
438     cout << "\nCodigo del producto: ";
439     cin >> codigo;
440
441     if(cin.fail()) {
442
443         cin.clear();
444         cin.ignore(1000, '\n');
445
446         cout << "Codigo invalido.\n";
447

```

```

448         return;
449     }
450
451     fstream archivo(
452         archivoProductos,
453         ios::binary |
454         ios::in |
455         ios::out
456     );
457
458     if(!archivo) {
459
460         cout << "Error al abrir archivo.\n";
461         return;
462     }
463
464     while(
465         archivo.read(
466             (char*)&p,
467             sizeof(Producto))
468     ) {
469
470         if(
471             p.codigo == codigo &&
472             p.activo
473         ) {
474
475             encontrado = true;
476
477             cout << "Stock actual: "
478                 << p.stock
479                 << endl;
480
481             cout << "Nuevo stock: ";

```

```

482         cin >> p.stock;
483
484         if(cin.fail() ||
485            p.stock < 0) {
486
487             cin.clear();
488             cin.ignore(1000, '\n');
489
490             cout << "Stock invalido.\n";
491
492             archivo.close();
493             return;
494         }
495
496         streampos posicion =
497             archivo.tellg();
498
499         archivo.seekp(
500             posicion -
501             static_cast<streamoff>(
502                 sizeof(Producto))
503         );
504
505         archivo.write(
506             (char*)&p,
507             sizeof(Producto)
508         );
509
510         cout
511         << "\nStock actualizado correctamente.\n";
512
513         break;

```

```

514     }
515 }
516
517     archivo.close();
518
519     if(!encontrado) {
520
521         cout << "\nProducto no encontrado.\n";
522     }
523 }
524
525 //===== MODIFICAR PRECIO =====//
526
527 void ModificarPrecio() {
528
529     Producto p;
530
531     int codigo;
532     bool encontrado = false;
533
534     cout << "\nCodigo del producto: ";
535     cin >> codigo;
536
537     if(cin.fail()) {
538
539         cin.clear();
540         cin.ignore(1000, '\n');
541
542         cout << "Codigo invalido.\n";
543         return;
544     }

```

```

545
546     fstream archivo(
547         archivoProductos,
548         ios::binary |
549         ios::in |
550         ios::out
551     );
552
553     if(!archivo) {
554
555         cout << "Error al abrir archivo.\n";
556         return;
557     }
558
559     while(
560         archivo.read(
561             (char*)&p,
562             sizeof(Producto))
563     ) {
564
565         if(
566             p.codigo == codigo &&
567             p.activo
568         ) {
569
570             encontrado = true;
571
572             cout << "Precio actual: Q"
573                 << p.precio
574                 << endl;
575
576             cout << "Nuevo precio: ";
577             cin >> p.precio;
578
579             if(cin.fail() ||

```

```

580         p.precio <= 0) {
581
582             cin.clear();
583             cin.ignore(1000, '\n');
584
585             cout << "Precio invalido.\n";
586
587             archivo.close();
588             return;
589         }
590
591         streampos posicion =
592             archivo.tellg();
593
594         archivo.seekp(
595             posicion -
596             static_cast<streamoff>(
597                 sizeof(Producto))
598             );
599
600         archivo.write(
601             (char*)&p,
602             sizeof(Producto)
603         );
604
605         cout
606         << "\nPrecio actualizado correctamente.\n";
607
608         break;
609     }
610 }
611
612 archivo.close();
613

```

```

614 if(!encontrado) {
615     cout << "\nProducto no encontrado.\n";
616 }
617 }
618
619 //===== ELIMINAR PRODUCTO =====//
620
621 void EliminarProducto() {
622
623     Producto p;
624
625     int codigo;
626     bool encontrado = false;
627
628     cout << "\nCodigo del producto: ";
629     cin >> codigo;
630
631     if(cin.fail()) {
632         cin.clear();
633         cin.ignore(1000, '\n');
634
635         cout << "Codigo invalido.\n";
636         return;
637     }
638
639     fstream archivo(
640         archivoProductos,
641         ios::binary |
642         ios::in |
643         ios::out
644     );
645
646
647

```

```

648     if(!archivo) {
649
650         cout << "Error al abrir archivo.\n";
651         return;
652     }
653
654     while(
655         archivo.read(
656             (char*)&p,
657             sizeof(Producto))
658     ) {
659
660         if(
661             p.codigo == codigo &&
662             p.activo
663         ) {
664
665             encontrado = true;
666
667             p.activo = false;
668
669             streampos posicion =
670                 archivo.tellg();
671
672             archivo.seekp(
673                 posicion -
674                 static_cast<streamoff>(
675                     sizeof(Producto))
676             );
677
678             archivo.write(
679                 (char*)&p,
680                 sizeof(Producto)
681             );
682
683             cout
684                 << "\nProducto eliminado correctamente.\n";
685
686             break;
687         }
688     }
689
690     archivo.close();
691
692     if(!encontrado) {
693
694         cout << "\nProducto no encontrado.\n";
695     }
696 }
697
698 //===== ORDENAR POR PRECIO =====//
699
700 void OrdenarPorPrecio() {
701
702     vector<Producto> productos =
703         cargarProductos();
704
705     if(productos.empty()) {
706
707         cout << "\nNo existen productos registrados.\n";
708         return;
709     }
710
711     // Bubble Sort Ascendente
712
713     for(size_t i = 0;

```

```

714     i < productos.size() - 1;
715     i++) {
716
717         for(size_t j = 0;
718             j < productos.size() - 1 - i;
719             j++) {
720
721             if(
722                 productos[j].precio >
723                 productos[j + 1].precio
724             ) {
725
726                 Producto aux =
727                     productos[j];
728
729                 productos[j] =
730                     productos[j + 1];
731
732                 productos[j + 1] =
733                     aux;
734             }
735         }
736     }
737
738     Encabezado(
739         "PRODUCTOS ORDENADOS POR PRECIO"
740     );
741
742     FormatoDecimal();
743
744     for(size_t i = 0;
745         i < productos.size();
746         i++) {
747
748         cout
749             << productos[i].codigo
750             << " | "
751             << productos[i].nombre
752             << " | 0"
753             << productos[i].precio
754             << endl;
755         }
756     }
757
758     //===== ORDENAR POR STOCK =====//
759
760     void OrdenarPorStock() {
761
762         vector<Producto> productos =
763             CargarProductos();
764
765         if(productos.empty()) {
766
767             cout << "\nNo existen productos registrados.\n";
768             return;
769         }
770
771         // Selection Sort
772
773         for(size_t i = 0;
774             i < productos.size() - 1;
775             i++) {
776
777             size_t menor = i;
778
779             for(size_t j = i + 1;
780                 j < productos.size();
781                 j++) {

```

```

783         productos[j].stock <
784         productos[menor].stock
785     ) {
786     }
787     menor = j;
788 }
789
790
791 Producto aux =
792 productos[i];
793
794 productos[i] =
795 productos[menor];
796
797 productos[menor] =
798 aux;
799 }
800
801 Encabezado(
802 "PRODUCTOS ORDENADOS POR STOCK"
803 );
804
805 for(size_t i = 0;
806 i < productos.size();
807 i++) {
808
809     cout
810     << productos[i].codigo
811     << " | "
812     << productos[i].nombre
813     << " | Stock: "
814     << productos[i].stock
815     << endl;
816 }
817 }
818
819 //===== ORDENAR POR VENDIDOS =====//
820
821 void OrdenarPorVendidos() {
822
823     vector<Producto> productos =
824     CargarProductos();
825
826     if(productos.empty()) {
827
828         cout << "\nNo existen productos registrados.\n";
829         return;
830     }
831
832     // Bubble Sort Descendente
833
834     for(size_t i = 0;
835 i < productos.size() - 1;
836 i++) {
837
838         for(size_t j = 0;
839 j < productos.size() - 1 - i;
840 j++) {
841
842             if(
843                 productos[j].vendidos <
844                 productos[j + 1].vendidos
845             ) {
846
847                 Producto aux =
848                 productos[j];
849
850                 productos[j] =
851                 productos[j + 1];
852
853                 productos[j + 1] =
854                 aux;
855             }
856         }
857     }
858
859     Encabezado(
860     "PRODUCTOS MAS VENDIDOS"
861     );
862
863     for(size_t i = 0;
864 i < productos.size();
865 i++) {
866
867         cout
868         << productos[i].codigo
869         << " | "
870         << productos[i].nombre
871         << " | Vendidos: "
872         << productos[i].vendidos
873         << endl;
874     }
875 }
876

```

Archivo ventas.h

```

C Ventas.h > ...
1 //===== ventas.h =====//
2
3 #ifndef VENTAS_H
4 #define VENTAS_H
5
6 //===== DETALLE VENTA =====//
7
8 struct DetalleVenta {
9
10     int codigoProducto;
11
12     char nombreProducto[40];
13
14     int cantidad;
15
16     float precioUnitario;
17
18     float subtotal;
19 };
20
21 //===== VENTA =====//
22
23 struct Venta {
24
25     int numeroVenta;
26
27     int dia;
28     int mes;
29     int anio;
30
31     int totalArticulos;
32
33     float subtotalGeneral;
34
35     float iva;
36
37     float descuento;
38
39     float totalFinal;
40 };
41
42 //===== PROTOTIPOS =====//
43
44 void CrearVenta();
45
46 #endif

```

Archivo ventas .cpp

```

1 //===== ventas.cpp =====//
2
3 #include <iostream>
4 #include <fstream>
5 #include <iomanip>
6 #include <vector>
7 #include <string>
8
9 #include "ventas.h"
10 #include "productos.h"
11 #include "utilidades.h"
12
13 using namespace std;
14
15 //===== ARCHIVOS =====//
16
17 const char* archivoProductosVentas =
18 "productos.dat";
19
20 const char* archivoVentas =
21 "ventas.dat";
22
23 //===== CREAR VENTA =====//
24
25 void CrearVenta() {
26
27     try {
28
29         Venta venta;
30
31         vector<DetalleVenta> detalles;
32
33         int cantidadProductos;
34
35         Encabezado("NUEVA VENTA");
36
37         cout << "Numero de venta: ";
38         cin >> venta.numeroVenta;
39
40         if(cin.fail()) {
41             throw 1;
42         }
43
44         //===== FECHA =====//
45
46         cout << "Dia: ";
47         cin >> venta.dia;
48
49         if(cin.fail() ||
50            venta.dia < 1 ||
51            venta.dia > 31) {
52
53             throw 2;
54         }
55
56         cout << "Mes: ";
57         cin >> venta.mes;
58
59         if(cin.fail() ||
60            venta.mes < 1 ||
61            venta.mes > 12) {
62
63             throw 3;
64         }
65
66         cout << "Año: ";
67         cin >> venta.ano;
68
69         if(cin.fail() ||
70            venta.ano < 2020) {
71

```



```

72         throw 4;
73     }
74
75     cout << "Cantidad de productos: ";
76     cin >> cantidadProductos;
77
78     if(cin.fail() ||
79        cantidadProductos <= 0) {
80
81         throw 5;
82     }
83
84     venta.subtotalGeneral = 0;
85     venta.totalArticulos = 0;
86
87     //===== RECORRER PRODUCTOS =====//
88
89     for(int i = 0;
90         i < cantidadProductos;
91         i++) {
92
93         Producto p;
94
95         DetalleVenta detalle;
96
97         bool encontrado = false;
98
99         int codigo;
100
101         cout
102         << "\nCodigo producto #"
103         << i + 1
104         << ": ";
105
106         cin >> codigo;
107
108         if(cin.fail()) {

```

```

109             throw 6;
110         }
111
112         fstream archivo(
113             archivoProductosVentas,
114             ios::binary |
115             ios::in |
116             ios::out
117         );
118
119         if(!archivo) {
120
121             cout
122             << "Error al abrir productos.dat\n";
123
124             return;
125         }
126
127         while(
128             archivo.read(
129                 (char*)&p,
130                 sizeof(Producto))
131         ) {
132
133             if(
134                 p.codigo == codigo &&
135                 p.activo
136             ) {
137
138                 encontrado = true;
139
140                 detalle.codigoProducto =
141                     p.codigo;
142
143                 strcpy(
144                     detalle.nombreProducto,
145                     p.nombre

```

```

146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181

```

```

    detalle.precioUnitario =
    p.precio;

    cout << "Cantidad: ";
    cin >> detalle.cantidad;

    if(
        cin.fail() ||
        detalle.cantidad <= 0
    ) {

        archivo.close();
        throw 7;
    }

    //===== VALIDAR STOCK =====//

    if(
        detalle.cantidad >
        p.stock
    ) {

        cout
        << "\nStock insuficiente.\n";

        archivo.close();
        return;
    }

    //===== CALCULAR SUBTOTAL =====//

    detalle.subtotal =
    detalle.cantidad *
    detalle.precioUnitario;

```

```

182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216

```

```

    //===== GUARDAR EN VECTOR =====//

    detalles.push_back(
    detalle);

    venta.subtotalGeneral +=
    detalle.subtotal;

    venta.totalArticulos +=
    detalle.cantidad;

    //===== ACTUALIZAR INVENTARIO =====//

    p.stock -=
    detalle.cantidad;

    p.vendidos +=
    detalle.cantidad;

    streampos posicion =
    archivo.tellg();

    archivo.seekp(
        posicion -
        static_cast<streamoff>(
            sizeof(Producto))
    );

    archivo.write(
        (char*)&p,
        sizeof(Producto)
    );

    break;
}

```

```

217     }
218
219     archivo.close();
220
221     if(!encontrado) {
222
223         cout
224         << "\nProducto no encontrado.\n";
225
226         return;
227     }
228
229 } // FIN DEL FOR DE PRODUCTOS
230
231 | //===== IVA =====//
232
233 venta.iva =
234 venta.subtotalGeneral * 0.12f;
235
236 //===== DESCUENTOS =====//
237
238 venta.descuento = 0;
239
240 // Descuento 5%
241 // si subtotal > Q500
242
243 if(
244     venta.subtotalGeneral >
245     500
246 ) {
247
248     venta.descuento +=
249     venta.subtotalGeneral *
250     0.05f;
251 }
252
253 // Bono adicional

```

```

254
255 if(
256     venta.totalArticulos % 5
257     == 0
258 ) {
259
260     venta.descuento += 25;
261 }
262
263 //===== TOTAL FINAL =====//
264
265 venta.totalFinal =
266
267 (
268     venta.subtotalGeneral +
269     venta.iva
270 )
271 -
272 venta.descuento;
273
274 //===== FACTURA =====//
275
276 Encabezado("FACTURA");
277
278 FormatoDecimal();
279
280 cout
281 << "\nNo. Venta: "
282 << venta.numeroVenta
283 << endl;
284
285 cout
286 << "Fecha: "
287 << venta.dia
288 << "/"
289 << venta.mes

```

```

290     << "/"
291     << venta.anio
292     << endl;
293
294     cout
295     << "\nPRODUCTOS\n";
296
297     cout
298     << "-----\n";
299
300     for(
301         size_t i = 0;
302         i < detalles.size();
303         i++
304     ) {
305
306         cout
307         << detalles[i].nombreProducto
308         << " x"
309         << detalles[i].cantidad
310         << " = Q"
311         << detalles[i].subtotal
312         << endl;
313     }
314
315     cout
316     << "-----\n";
317
318     cout
319     << "Subtotal: Q"
320     << venta.subtotalGeneral
321     << endl;
322
323     cout
324     << "IVA (12%): Q"
325     << venta.iva
326     << endl;
327
328     cout
329     << "Descuento: Q"
330     << venta.descuento
331     << endl;
332
333     cout
334     << "TOTAL: Q"
335     << venta.totalFinal
336     << endl;
337
338     //===== GUARDAR VENTA =====//
339
340     ofstream guardar(
341         archivoVentas,
342         ios::binary | ios::app
343     );
344
345     if(!guardar) {
346
347         cout
348         << "\nError al abrir ventas.dat\n";
349
350         return;
351     }
352
353     guardar.write(
354         (char*)&venta,
355         sizeof(venta)
356     );
357
358     guardar.close();
359
360     cout
361     << "\nVenta almacenada correctamente.\n";

```

```

362     }
363
364     //===== MANEJO DE EXCEPCIONES =====//
365
366     catch(int error) {
367
368         cin.clear();
369         cin.ignore(1000, '\n');
370
371         cout
372         << "\nError en los datos ingresados.\n";
373
374         switch(error) {
375
376             case 1:
377                 cout
378                 << "Numero de venta invalido.\n";
379                 break;
380
381             case 2:
382                 cout
383                 << "Dia invalido.\n";
384                 break;
385
386             case 3:
387                 cout
388                 << "Mes invalido.\n";
389                 break;
390
391             case 4:
392                 cout
393                 << "Año invalido.\n";
394                 break;
395
396             case 5:
397                 cout
398                 << "Cantidad de productos invalida.\n";
399                 break;
400
401             case 6:
402                 cout
403                 << "Codigo de producto invalido.\n";
404                 break;
405
406             case 7:
407                 cout
408                 << "Cantidad invalida.\n";
409                 break;
410
411             default:
412                 cout
413                 << "Error desconocido.\n";
414             }
415         }
416     }

```

Archivo Reportes .h

```

1 //===== reportes.h =====//
2
3 #ifndef REPORTES_H
4 #define REPORTES_H
5
6 //===== MENU REPORTES =====//
7
8 void MenuReportes();
9
10 //===== REPORTES =====//
11
12 void ReporteStockBajo();
13
14 void ProductosMasVendidos();
15
16 void VentasTotales();
17
18 void VentasDelDia();
19
20 void VentasPorMes();
21
22 void EstadisticasGenerales();
23
24 void ExportarReporteTXT();
25
26 #endif

```

Archivo Reportes .cpp

```

Reportes.cpp > ExportarReporteTXT
1 //===== reportes.cpp =====//
2
3 #include <iostream>
4 #include <fstream>
5 #include <iomanip>
6 #include <vector>
7
8 #include "reportes.h"
9 #include "productos.h"
10 #include "ventas.h"
11 #include "utilidades.h"
12
13 using namespace std;
14
15 //===== ARCHIVOS =====//
16
17 const char* archivoVentasReporte =
18 "ventas.dat";
19
20 //===== MATRIZ =====//
21
22 float ventasPorMes[12][31] = {0};
23
24 //===== MENU REPORTES =====//
25
26 void MenuReportes() {
27     int opcion;
28
29     do {
30
31         Encabezado("MODULO REPORTES");
32
33
34         cout << "1. Productos con stock bajo\n";
35         cout << "2. Productos mas vendidos\n";
36         cout << "3. Ventas totales\n";
37         cout << "4. Ventas del dia\n";
38
39         cout << "5. Ventas por mes\n";
40         cout << "6. Estadisticas generales\n";
41         cout << "7. Exportar reporte TXT\n";
42         cout << "8. Regresar\n";
43
44         cout << "\nseleccione opcion: ";
45         cin >> opcion;
46
47         if(cin.fail()) {
48             cin.clear();
49             cin.ignore(1000, '\n');
50
51             opcion = 0;
52         }
53
54         switch(opcion) {
55
56             case 1:
57                 ReporteStockBajo();
58                 break;
59
60             case 2:
61                 ProductosMasVendidos();
62                 break;
63
64             case 3:
65                 VentasTotales();
66                 break;
67
68             case 4:
69                 VentasDelDia();
70                 break;
71

```

```

72         case 5:
73             VentasPorMes();
74             break;
75
76         case 6:
77             EstadisticasGenerales();
78             break;
79
80         case 7:
81             ExportarReporteTXT();
82             break;
83
84         case 8:
85             break;
86
87         default:
88             cout << "\nOpcion invalida.\n";
89     }
90
91     if(opcion != 8) {
92         PausarSistema();
93         LimpiarPantalla();
94     }
95 } while(opcion != 8);
96
97 //===== STOCK BAJO =====//
98
99 void ReporteStockBajo() {
100     vector<Producto> productos =
101     cargarProductos();

```

```

102
103     if(productos.empty()) {
104         cout << "\nNo existen productos registrados.\n";
105         return;
106     }
107
108     Encabezado("PRODUCTOS CON STOCK BAJO");
109
110     bool encontrado = false;
111
112     for(size_t i = 0;
113         i < productos.size();
114         i++) {
115         if(productos[i].stock <= 5) {
116             encontrado = true;
117
118             cout
119             << productos[i].codigo
120             << " | "
121             << productos[i].nombre
122             << " | Stock: "
123             << productos[i].stock
124             << endl;
125         }
126     }
127
128     if(!encontrado) {
129         cout
130         << "\nNo existen productos con stock bajo.\n";
131     }

```

```

138     }
139 }
140
141 //===== PRODUCTOS MAS VENDIDOS =====//
142
143 void ProductosMasVendidos() {
144
145     vector<Producto> productos =
146     CargarProductos();
147
148     if(productos.empty()) {
149
150         cout << "\nNo existen productos registrados.\n";
151         return;
152     }
153
154     // Bubble Sort Descendente
155
156     for(size_t i = 0;
157         i < productos.size() - 1;
158         i++) {
159
160         for(size_t j = 0;
161             j < productos.size() - 1 - i;
162             j++) {
163
164             if(
165                 productos[j].vendidos <
166                 productos[j + 1].vendidos
167             ) {
168
169                 Producto aux =
170                 productos[j];
171

```

```

172                 productos[j] =
173                 productos[j + 1];
174
175                 productos[j + 1] =
176                 aux;
177             }
178         }
179     }
180
181     Encabezado("PRODUCTOS MAS VENDIDOS");
182
183     for(size_t i = 0;
184         i < productos.size();
185         i++) {
186
187         cout
188         << productos[i].codigo
189         << " | "
190         << productos[i].nombre
191         << " | Vendidos: "
192         << productos[i].vendidos
193         << endl;
194     }
195 }
196 //===== VENTAS TOTALES =====//
197
198 void VentasTotales() {
199
200     Venta v;
201
202     float totalVentas = 0;
203

```

```

204     int cantidadVentas = 0;
205
206     ifstream archivo(
207         archivoVentasReporte,
208         ios::binary
209     );
210
211     if(!archivo) {
212
213         cout << "\nNo existen ventas registradas.\n";
214         return;
215     }
216
217     while(
218         archivo.read(
219             (char*)&v,
220             sizeof(Venta)
221         ) {
222
223             totalVentas +=
224             v.totalFinal;
225
226             cantidadVentas++;
227         }
228
229     archivo.close();
230
231     Encabezado("VENTAS TOTALES");
232
233     FormatoDecimal();
234
235     cout
236     << "Cantidad de ventas: "
237     << cantidadVentas

```



```

238     << endl;
239
240     cout
241     << "Total vendido: Q"
242     << totalVentas
243     << endl;
244 }
245
246 //===== VENTAS DEL DIA =====//
247
248 void VentasDelDia() {
249
250     int diaBuscar;
251     int mesBuscar;
252     int anioBuscar;
253
254     cout << "\nDia: ";
255     cin >> diaBuscar;
256
257     cout << "Mes: ";
258     cin >> mesBuscar;
259
260     cout << "Anio: ";
261     cin >> anioBuscar;
262
263     Venta v;
264
265     float totalDia = 0;
266
267     int ventasDia = 0;
268
269     ifstream archivo(
270         archivoVentasReporte,
271         ios::binary

```

```

272     );
273
274     if(!archivo) {
275
276         cout << "\nNo existen ventas registradas.\n";
277         return;
278     }
279
280     while(
281         archivo.read(
282             (char*)&v,
283             sizeof(Venta))
284     ) {
285
286         if(
287             v.dia == diaBuscar &&
288             v.mes == mesBuscar &&
289             v.anio == anioBuscar
290         ) {
291
292             totalDia +=
293             v.totalFinal;
294
295             ventasDia++;
296         }
297     }
298
299     archivo.close();
300
301     Encabezado("VENTAS DEL DIA");
302
303     FormatoDecimal();
304
305     cout

```

```

306     << "Cantidad de ventas: "
307     << ventasDia
308     << endl;
309
310     cout
311     << "Total vendido: Q"
312     << totalDia
313     << endl;
314 }
315 //===== VENTAS POR MES =====//
316
317 void VentasPorMes() {
318
319     Venta v;
320
321     // Reiniciar matriz
322
323     for(int i = 0; i < 12; i++) {
324
325         for(int j = 0; j < 31; j++) {
326
327             ventasPorMes[i][j] = 0;
328         }
329     }
330
331     ifstream archivo(
332         archivoVentasReporte,
333         ios::binary
334     );
335
336     if(!archivo) {
337
338         cout << "\nNo existen ventas registradas.\n";
339         return;

```

```

340     }
341
342     while(
343         archivo.read(
344             (char*)&v,
345             sizeof(Venta))
346     ) {
347
348         if(
349             v.mes >= 1 &&
350             v.mes <= 12 &&
351             v.dia >= 1 &&
352             v.dia <= 31
353         ) {
354
355             ventasPorMes
356             [v.mes - 1]
357             [v.dia - 1]
358
359             +=
360
361             v.totalFinal;
362         }
363     }
364
365     archivo.close();
366
367     Encabezado("VENTAS POR MES");
368
369     FormatoDecimal();
370
371     for(int mes = 0;
372         mes < 12;
373         mes++) {
374

```

```

375     float totalMes = 0;
376
377     for(int dia = 0;
378         dia < 31;
379         dia++) {
380
381         totalMes +=
382             ventasPorMes[mes][dia];
383     }
384
385     cout
386     << "Mes "
387     << mes + 1
388     << ": Q"
389     << totalMes
390     << endl;
391 }
392 }
393
394 //===== ESTADISTICAS GENERALES =====//
395
396 void EstadisticasGenerales() {
397
398     vector<Producto> productos =
399         CargarProductos();
400
401     Venta v;
402
403     int totalProductos =
404
405     productos.size();
406
407     int stockTotal = 0;
408

```

```

409 float totalVentas = 0;
410
411 int cantidadVentas = 0;
412
413 //===== PRODUCTOS =====//
414
415 for(size_t i = 0;
416     i < productos.size();
417     i++) {
418
419     stockTotal +=
420     productos[i].stock;
421 }
422
423 //===== VENTAS =====//
424
425 ifstream archivo(
426     archivoVentasReporte,
427     ios::binary
428 );
429
430 while(
431     archivo.read(
432         (char*)&v,
433         sizeof(Venta))
434 ) {
435
436     totalVentas +=
437     v.totalFinal;
438
439     cantidadVentas++;
440 }
441
442 archivo.close();

```

```

443 //===== MOSTRAR =====//
444
445 Encabezado(
446 "ESTADISTICAS GENERALES"
447 );
448
449 FormatoDecimal();
450
451 cout
452 << "Productos registrados: "
453 << totalProductos
454 << endl;
455
456 cout
457 << "Stock total: "
458 << stockTotal
459 << endl;
460
461 cout
462 << "Cantidad ventas: "
463 << cantidadVentas
464 << endl;
465
466 cout
467 << "Total vendido: Q"
468 << totalVentas
469 << endl;
470
471 }
472 //===== EXPORTAR REPORTE TXT =====//
473
474 void ExportarReporteTXT() {
475
476     vector<Producto> productos =

```

```

477     CargarProductos();
478
479     Venta v;
480
481     int cantidadVentas = 0;
482
483     float totalVentas = 0;
484
485     ofstream reporte(
486         "reporte_general.txt"
487     );
488
489     if(!reporte) {
490
491         cout
492         << "\nNo fue posible crear el reporte.\n";
493
494         return;
495     }
496
497     //===== PRODUCTOS =====//
498
499     reporte
500     << "=====\n";
501
502     reporte
503     << "    REPORTE GENERAL DEL SISTEMA\n";
504
505     reporte
506     << "=====\n\n";
507
508     reporte

```

```

509 << "PRODUCTOS REGISTRADOS\n\n";
510
511 reporte
512 << left
513 << setw(10)
514 << "Codigo"
515
516 << setw(25)
517 << "Nombre"
518
519 << setw(12)
520 << "Precio"
521
522 << setw(10)
523 << "Stock"
524
525 << setw(10)
526 << "Vendidos"
527
528 << endl;
529
530 reporte
531 << "-----\n";
532
533 for(size_t i = 0;
534     i < productos.size();
535     i++) {
536
537     reporte
538     << left
539
540     << setw(10)
541     << productos[i].codigo
542

```

```

543     << setw(25)
544     << productos[i].nombre
545
546     << setw(12)
547     << fixed
548     << setprecision(2)
549     << productos[i].precio
550
551     << setw(10)
552     << productos[i].stock
553
554     << setw(10)
555     << productos[i].vendidos
556
557     << endl;
558 }
559
560 //===== VENTAS =====//
561
562 ifstream archivo(
563     archivoVentasReporte,
564     ios::binary
565 );
566
567 while(
568     archivo.read(
569         (char*)&v,
570         sizeof(Venta))
571 ) {
572
573     totalVentas +=
574     v.totalFinal;
575
576     cantidadVentas++;
577 }
578
579 archivo.close();

```

```

580
581     reporte
582     << "\n\n===== \n";
583
584     reporte
585     << "RESUMEN DE VENTAS\n";
586
587     reporte
588     << "===== \n\n";
589
590     reporte
591     << "Cantidad de ventas: "
592     << cantidadVentas
593     << endl;
594
595     reporte
596     << "Total vendido: Q"
597     << fixed
598     << setprecision(2)
599     << totalVentas
600     << endl;
601
602     reporte.close();
603
604     cout
605     << "\nReporte exportado correctamente.\n";
606
607     cout
608     << "Archivo generado: reporte_general.txt\n";
609 }

```

Archivo Utilidades .h

```

C Utilidades.h > ...
1 //===== utilidades.h =====//
2
3 #ifndef UTILIDADES_H
4 #define UTILIDADES_H
5
6 #include <string>
7
8 using namespace std;
9
10 //===== FORMATO =====//
11
12 void Encabezado(string titulo);
13
14 void FormatoDecimal();
15
16 //===== SISTEMA =====//
17
18 void PausarSistema();
19
20 void LimpiarPantalla();
21
22 //===== UTILIDADES =====//
23
24 void MenuUtilidades();
25
26 void ReiniciarInventario();
27
28 #endif

```

Archivo Utilidad .cpp

```

1 //===== utilidades.cpp =====//
2
3 #include <iostream>
4 #include <iomanip>
5 #include <fstream>
6 #include <cstdlib>
7
8 #include "utilidades.h"
9 #include "productos.h"
10
11 using namespace std;
12
13 //===== ENCABEZADO =====//
14
15 void Encabezado(string titulo) {
16     cout << "\n=====n";
17     cout << " " << titulo << endl;
18     cout << "=====n";
19 }
20
21 //===== FORMATO DECIMAL =====//
22
23 void FormatoDecimal() {
24     cout << fixed << setprecision(2);
25 }
26
27 //===== PAUSAR SISTEMA =====//
28
29 void PausarSistema() {
30     cin.ignore(1000, '\n');
31     cout << "\nPresione ENTER para continuar...";
32     cin.get();
33 }

```

```

38 //===== LIMPIAR PANTALLA =====//
39
40 void LimpiarPantalla() {
41     #ifdef _WIN32
42         system("cls");
43     #else
44         system("clear");
45     #endif
46 }
47
48 //===== REINICIAR INVENTARIO =====//
49
50 void ReiniciarInventario() {
51     char respuesta;
52
53     cout << "\nADVERTENCIA\n";
54     cout << "Esta accion eliminara todos los productos.\n";
55     cout << "Desea continuar? (S/N): ";
56     cin >> respuesta;
57
58     respuesta = toupper(respuesta);
59
60     if(respuesta == 'S') {
61         ofstream archivo(
62             "productos.dat",
63             ios::binary | ios::trunc
64         );
65         archivo.close();
66     }
67 }
68
69
70
71

```

```

72         cout << "\nInventario reiniciado correctamente.\n";
73     }
74     else {
75     }
76         cout << "\nOperacion cancelada.\n";
77     }
78 }
79
80 //===== MENU UTILIDADES =====//
81
82 void MenuUtilidades() {
83
84     int opcion;
85
86     do {
87
88         Encabezado("UTILIDADES DEL SISTEMA");
89
90         cout << "1. Reiniciar inventario\n";
91         cout << "2. Regresar\n";
92
93         cout << "\nSeleccione una opcion: ";
94         cin >> opcion;
95
96         if(cin.fail()) {
97
98             cin.clear();
99             cin.ignore(1000, '\n');
100
101             opcion = 0;
102         }
103
104         switch(opcion) {
105
106             case 1:
107
108                 ReiniciarInventario();
109                 break;
110
111             case 2:
112
113                 break;
114
115             default:
116
117                 cout << "\nOpcion invalida.\n";
118             }
119
120             if(opcion != 2) {
121
122                 PausarSistema();
123                 LimpiarPantalla();
124             }
125
126         } while(opcion != 2);
127     }
128 }

```

Programa Ejecutándose

```

EDITORES ABIERTOS
C: Main.cpp
C: Productos.h
C: Productos.cpp
C: Ventas.h
C: Ventas.cpp
C: Reportes.h
C: Reportes.cpp
C: Utilidades.h
C: Utilidades.cpp

BUSCAR
Reemplazar

PROYECTO FINAL
Evidencia.docx
Evidencia Avance ...
Main.cpp
Productos.cpp
Productos.h

ESQUEMA
opcionesMenu
MenuPrincipal()
main()

C: Main.cpp > main()
MenuReporte
break;

case 4:
MenuUtil
break;

case 5:
cout <<
break;

default:
cout <<

if(opcion != 5)
PausarSistem
LimpiarPanta
}

while(opcion != 5)

return 0;
}

```

Prueba


```
C:\Users\vizita\OneDrive\Docu x + v

=====
      SISTEMA DE INVENTARIO Y VENTAS
=====
1. Gestion de Productos
2. Proceso de Venta
3. Reportes
4. Utilidades
5. Salir

Seleccione una opcion: 1

=====
      GESTION DE PRODUCTOS
=====
1. Registrar producto
2. Mostrar productos
3. Buscar por codigo
4. Buscar por nombre
5. Actualizar stock
6. Modificar precio
7. Eliminar producto
8. Ordenar por precio
9. Ordenar por stock
10. Ordenar por vendidos
11. Regresar

Seleccione opcion: 1

Codigo: 004
```

```
C:\Users\vizita\OneDrive\Docu x + v

Nombre: Album Mundial Panini (Pasta Dura)
Precio: 150
Stock: 20

Producto registrado correctamente.

=====
      GESTION DE PRODUCTOS
=====
1. Registrar producto
2. Mostrar productos
3. Buscar por codigo
4. Buscar por nombre
5. Actualizar stock
6. Modificar precio
7. Eliminar producto
8. Ordenar por precio
9. Ordenar por stock
10. Ordenar por vendidos
11. Regresar

Seleccione opcion: 1

Codigo: 005
Nombre: Caja de Sobres
Precio: 975
Stock: 15

Producto registrado correctamente.
```

```
C:\Users\izita\OneDrive\Docu x + v

=====
GESTION DE PRODUCTOS
=====
1. Registrar producto
2. Mostrar productos
3. Buscar por codigo
4. Buscar por nombre
5. Actualizar stock
6. Modificar precio
7. Eliminar producto
8. Ordenar por precio
9. Ordenar por stock
10. Ordenar por vendidos
11. Regresar

Seleccione opcion: 2

=====
LISTA DE PRODUCTOS
=====
Codigo  Nombre                Precio  Stock  Vendidos
-----
1      Forza Horizon           250.00  10     0
2      Super Smash Bros Ultimate 600.00  25     0
3      Resident Evil 4 Remake    450.00  15     0
4      Album Mundial Panini (Pasta Dura) 150.00  20     0
5      Caja de Sobres           975.00  15     0

=====
GESTION DE PRODUCTOS
=====
```

```
C:\Users\izita\OneDrive\Docu x + v

=====
SISTEMA DE INVENTARIO Y VENTAS
=====
1. Gestion de Productos
2. Proceso de Venta
3. Reportes
4. Utilidades
5. Salir

Seleccione una opcion: 2

=====
NUEVA VENTA
=====
Numero de venta: 001
Dia: 29
Mes: 05
Año: 2026
Cantidad de productos: 2

Codigo producto #1: 004
Cantidad: 1

Codigo producto #2: 005
Cantidad: 1

=====
FACTURA
=====
```

```
=====
FACTURA
=====
No. Venta: 1
Fecha: 29/5/2026

PRODUCTOS
-----
Album Mundial Panini (Pasta Dura) x1 = Q150.00
Caja de Sobres x1 = Q975.00
-----
Subtotal: Q1125.00
IVA (12%): Q135.00
Descuento: Q56.25
TOTAL: Q1203.75

Venta almacenada correctamente.

Presione ENTER para continuar...|
```

```
C:\Users\izita\OneDrive\Docu x + v

=====
SISTEMA DE INVENTARIO Y VENTAS
=====
1. Gestion de Productos
2. Proceso de Venta
3. Reportes
4. Utilidades
5. Salir

Seleccione una opcion: 3

=====
MODULO REPORTES
=====
1. Productos con stock bajo
2. Productos mas vendidos
3. Ventas totales
4. Ventas del dia
5. Ventas por mes
6. Estadisticas generales
7. Exportar reporte TXT
8. Regresar

Seleccione opcion: 6

=====
ESTADISTICAS GENERALES
=====
Productos registrados: 5
Stock total: 83
Cantidad ventas: 1
Total vendido: Q1203.75

Presione ENTER para continuar...|
```

```
C:\Users\izita\OneDrive\Docu x + v

=====
SISTEMA DE INVENTARIO Y VENTAS
=====
1. Gestion de Productos
2. Proceso de Venta
3. Reportes
4. Utilidades
5. Salir

Seleccione una opcion: 2

=====
NUEVA VENTA
=====
Numero de venta: 002
Dia: 29
Mes: 05
Año: 2026
Cantidad de productos: 3

Codigo producto #1: 005
Cantidad: 1

Codigo producto #2: 004
Cantidad: 1

Codigo producto #3: 001
Cantidad: 1

=====
FACTURA
=====

No. Venta: 2
```

```
=====
FACTURA
=====

No. Venta: 2
Fecha: 29/5/2026

PRODUCTOS
=====
Caja de Sobres x1 = Q975.00
Album Mundial Panini (Pasta Dura) x1 = Q150.00
Forza Horizon x1 = Q250.00
=====
Subtotal: Q1375.00
IVA (12%): Q165.00
Descuento: Q68.75
TOTAL: Q1471.25

Venta almacenada correctamente.

Presione ENTER para continuar...|
```